

Student Score Filter

Question 1: 2025 Jan OPPE 1 - Set 1 – Section 3

Student Score Filter

Write a function that filters students based on performance criteria

```
def filter_students(data: dict, criteria: str) -> set:
    """Filter students based on score criteria

    Args:
        data: Dictionary with student names as keys and list of
        scores as values
        criteria: String specifying the filter criteria

    Returns:
        Set of student names matching the given criteria
    """

    pass # Your code here
```

The Four Criteria



Excellent

Average score ≥ 85

Top performers with consistently high grades



Good

$50 \leq \text{Average} < 85$

Decent average score but doesn't reach excellent threshold



All Pass

Every score ≥ 50

No failing grades in any assessment



Balanced

$\text{Max} - \text{Min} \leq 10$

Consistent performance across all tests

Example Data

The function will process this dictionary containing student names and their test scores:

```
data = {  
    "Alice": [90, 80, 85],  
    "Bob": [40, 50, 60],  
    "Charlie": [30, 40, 20],  
    "Ram": [78, 92, 85, 79, 81],  
    "Babu": [67, 70, 75],  
    "Kumar": [100, 100, 100, 100, 100, 100, 40]  
}
```

Each student has taken multiple tests, and we want to filter them based on different performance patterns.

Understand Each Condition Carefully

1

Excellent

Condition: $\text{Average} \geq 85$

Calculation: `sum(scores) / len(scores)`

Compute the mean of all scores and check if it meets the threshold.

Don't recalculate in the condition—store it in a variable.

2

Good

Condition: $50 \leq \text{Average} < 85$

Important: Include 50 as a passing score, but exclude 85 (since 85+ is excellent).

Must use `50 <= avg < 85` — not `avg > 50`.

3

All Pass

Condition: Every score ≥ 50

Use: `all(score >= 50 for score in scores)`

This built-in returns `True` only if all scores pass the condition.

4

Balanced

Condition: $\text{Max} - \text{Min} \leq 10$

Use: `max(scores) - min(scores) <= 10`

Calculates the range of scores to check consistency.

Step-by-Step Strategy



Create empty result set

Initialize an empty set to collect matching student names



Loop through dictionary

Iterate over each student and their scores using `.items()`



Compute required values

Calculate average, minimum, and maximum scores once per student



Apply condition based on criteria

Use if-elif statements to check the selected criteria



Add student name if condition passes

Include the student in the result set when criteria is met



Return set

Return the collection of matching student names

Clean and Safe Solution

```
def filter_students(data: dict, criteria: str) -> set:
    result = set()

    for name, scores in data.items():
        avg = sum(scores) / len(scores)

        if criteria == "excellent":
            if avg >= 85:
                result.add(name)
        elif criteria == "good":
            if 50 <= avg < 85:
                result.add(name)
        elif criteria == "all_pass":
            if all(score >= 50 for score in scores):
                result.add(name)
        elif criteria == "balanced":
            if max(scores) - min(scores) <= 10:
                result.add(name)

    return result
```



Why This Works

- Uses `.items()` to unpack
- Calculates average once
- Uses built-in functions
- Separates logic cleanly

Why This Structure Is Correct



Loop runs only once

Iterates through the data exactly one time, avoiding redundant processing



Average computed once

Calculates the average score per student outside the condition checks



Criteria checked cleanly

Uses if-elif chain to ensure only one condition is evaluated at a time



Independent conditions

Each condition is self-contained and doesn't interfere with others

Common Mistakes Students Make

✗ Forgetting to return a set

Returning a list instead of a set misses the requirement. Sets automatically handle duplicates and use `set()` for initialization, not `[]`.

✗ Multiple loops for each criteria

Writing separate loops for each condition is inefficient and causes code duplication

✗ Wrong comparison operators

Using `>` instead of `>=` for inclusive bounds

✗ Mixing inclusive/exclusive bounds

Example: `avg > 50` and `avg <= 85` is wrong for "good"—it excludes 50

✗ Modifying data during iteration

Never modify the dictionary while looping through it

✗ Nested condition confusion

Writing complex nested if statements that are hard to debug

Engineering Best Practices

01

Use `.items()` to unpack key and value

Avoid `data[key]` syntax when looping

02

Compute average once per student

Store `sum(scores) / len(scores)` in a variable

03

Use built-ins: `all()`, `max()`, `min()`

Leverage Python's optimized functions

04

Keep condition checks separate

Use if-elif chain, don't mix criteria logic

Logic Summary

1

Loop through students

Iterate over data dictionary

2

Check selected criteria

Evaluate condition based on input

3

Add matching names

Include students that meet criteria

4

Return set

Return collection of names